

9. Simulation of Network Attacks: Demonstrate basic attacks such as DoS or spoofing in a controlled environment .

1. Aim

To understand and simulate basic network attacks such as **Denial-of-Service (DoS)** and **IP spoofing** in a controlled Python environment using Google Colab, and to implement basic attack detection, traffic analysis, logging, and mitigation techniques.

2. Objectives

After completing this practical, students will be able to:

1. Understand the concept of network attacks.
 2. Understand Denial-of-Service (DoS) attacks.
 3. Understand IP spoofing attacks.
 4. Differentiate between normal and suspicious network traffic.
 5. Generate simulated network traffic using Python.
 6. Identify excessive traffic generated toward a destination.
 7. Detect suspicious or spoofed source IP addresses.
 8. Apply threshold-based attack detection.
 9. Generate security logs for detected attacks.
 10. Calculate normal and malicious traffic statistics.
 11. Apply basic mitigation decisions.
 12. Visualize simulated attack traffic.
 13. Execute the complete network attack simulation in Google Colab.
-

3. Introduction

A **network attack** is a deliberate attempt to disrupt, manipulate, intercept, or gain unauthorized access to network resources.

Common network attacks include:

- Denial-of-Service (DoS)
- Distributed Denial-of-Service (DDoS)
- IP spoofing
- Port scanning
- Man-in-the-Middle attacks
- Packet sniffing
- Network reconnaissance

In this practical, two basic attack scenarios are simulated:

1. DoS Attack

A **Denial-of-Service (DoS)** attack attempts to make a service or system unavailable by generating excessive requests or traffic.

In this practical, a DoS attack is represented by an unusually large number of simulated requests sent toward the same destination.

2. IP Spoofing

IP spoofing occurs when an attacker uses a forged source IP address to make traffic appear to originate from another system.

For example:

Actual Source → 10.0.0.25

Claimed Source IP → 192.168.1.100

The receiving system sees the claimed address rather than the actual source identity.

4. Controlled Attack Simulation

The practical does **not** generate real attack traffic.

Instead, Python creates simulated records such as:

Source IP	Destination IP	Requests
10.0.0.10	10.0.0.50	5
10.0.0.11	10.0.0.50	7
10.0.0.20	10.0.0.50	120

The monitoring system analyzes these records and determines whether the traffic is:

NORMAL

or

SUSPICIOUS / ATTACK

This approach is particularly suitable for Google Colab because the previous practical already uses a Python-based security simulation rather than an operating-system firewall.

5. Denial-of-Service Attack

A **DoS attack** attempts to exhaust a system's resources by sending an excessive number of requests or packets.

Normal Traffic

Client → Server

5 requests

10 requests

8 requests

Suspicious Traffic

Attacker → Server

500 requests

700 requests

900 requests

A simple detection mechanism can define a threshold:

Requests ≤ 50 → NORMAL

Requests > 50 → POSSIBLE DoS

The threshold is only for educational simulation and does not represent a universal real-world DoS threshold.

6. IP Spoofing

An IP address identifies the apparent source of network traffic.

In spoofing, an attacker may use an unauthorized or forged source address.

For this simulation, a list of trusted source IP addresses is maintained:

10.0.0.10

10.0.0.11

10.0.0.12

If traffic arrives claiming to originate from an unexpected address:

192.168.1.100

172.16.0.25

the system marks the traffic as:

SUSPICIOUS SOURCE

7. Attack Detection Model

The Python program uses two basic detection rules.

Rule 1 – DoS Detection

IF request_count > threshold

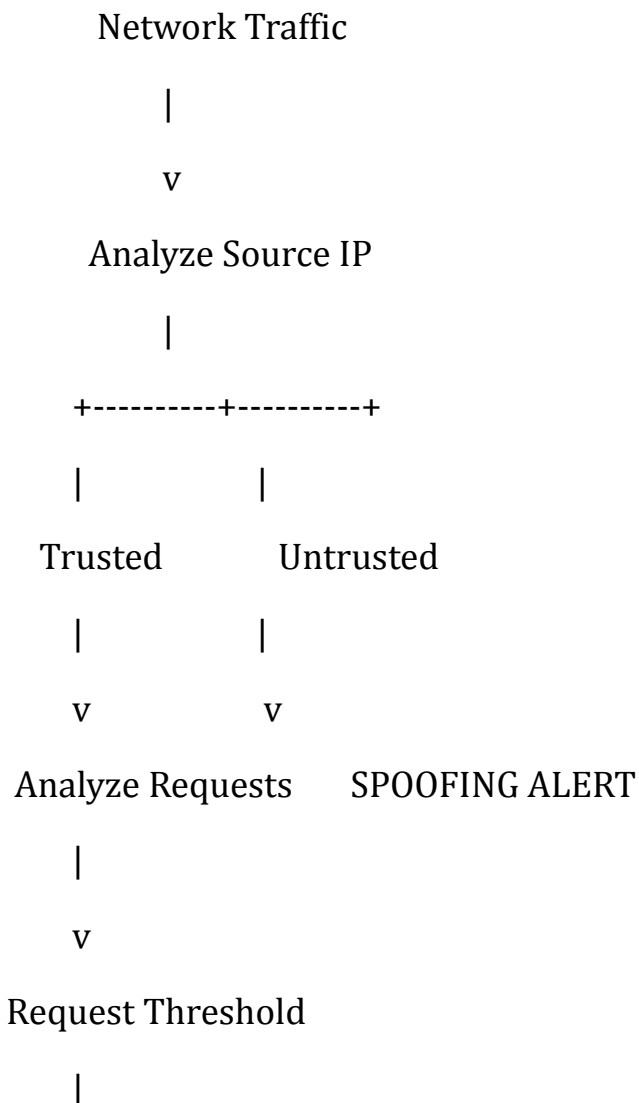
THEN possible DoS attack

Rule 2 – Spoofing Detection

IF source_IP is not in trusted_IP_list

THEN possible spoofing

Combined Detection



```

+-----+-----+
|       |
Normal   Excessive
|       |
v       v
NORMAL   DoS ALERT

```

8. Google Colab Requirements

Platform

Google Colab

Programming Language

Python

External Libraries

Only commonly available Python libraries are used:

random

datetime

collections

pandas

matplotlib

No real network interface, administrator privileges, packet injection, or external attack tools are required.

9. Complete Google Colab Program

Students can copy the following code into **one Google Colab cell** and execute it.

```

# =====
# PRACTICAL 9

```

```
# SIMULATION OF NETWORK ATTACKS
```

```
# =====
```

```
# Platform : Google Colab
```

```
# Language : Python
```

```
# Type : Controlled Network Attack Simulation
```

```
# Attacks : DoS and IP Spoofing
```

```
# =====
```

```
import random
```

```
from datetime import datetime
```

```
from collections import Counter
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# =====
```

```
# 1. CONFIGURATION
```

```
# =====
```

```
print("=" * 80)
```

```
print("      SIMULATION OF NETWORK ATTACKS")
```

```
print("    CONTROLLED DoS AND IP SPOOFING SIMULATION")
```

```
print("=" * 80)
```

```
# Trusted IP addresses
```

```
trusted_ips = [
```

```
    "10.0.0.10",
```

```
    "10.0.0.11",
```

```
    "10.0.0.12",
```

```
    "10.0.0.13"
```

```
]
```

```

# Destination server

server_ip = "10.0.0.50"


# DoS detection threshold

DOS_THRESHOLD = 50

print("\nTrusted IP Addresses:")

for ip in trusted_ips:

    print(" -", ip)

print("\nProtected Server:", server_ip)

print("DoS Detection Threshold:", DOS_THRESHOLD)

# =====

# 2. GENERATE NORMAL TRAFFIC

# =====

normal_traffic = []

normal_ips = trusted_ips

for i in range(30):

    source_ip = random.choice(normal_ips)

    request_count = random.randint(5, 20)

    normal_traffic.append({

        "source_ip": source_ip,

        "destination_ip": server_ip,

        "requests": request_count,

        "traffic_type": "NORMAL"

    })

# =====

# 3. GENERATE SIMULATED DoS TRAFFIC

```

```

# =====

dos_traffic = []

for i in range(8):
    source_ip = "10.0.0.99"
    request_count = random.randint(80, 150)
    dos_traffic.append({
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "requests": request_count,
        "traffic_type": "SIMULATED_DOS"
    })

# =====

# 4. GENERATE SIMULATED SPOOFED TRAFFIC

# =====

spoofed_traffic = []

spoofed_ips = [
    "192.168.1.200",
    "172.16.0.25",
    "192.0.2.55",
    "198.51.100.25"
]

for i in range(8):
    source_ip = random.choice(spoofed_ips)
    request_count = random.randint(10, 30)
    spoofed_traffic.append({
        "source_ip": source_ip,

```



```

        "destination_ip": server_ip,

        "requests": request_count,

        "traffic_type": "SIMULATED_SPOOFING"

    })

# =====

# 5. COMBINE TRAFFIC

# =====

traffic = normal_traffic + dos_traffic + spoofed_traffic

random.shuffle(traffic)

df = pd.DataFrame(traffic)

print("\nTotal Simulated Traffic Records:", len(df))

# =====

# 6. DISPLAY TRAFFIC

# =====


print("\n" + "=" * 80)

print("SIMULATED NETWORK TRAFFIC")

print("=" * 80)

print(df.to_string(index=False))

# =====

# 7. ATTACK DETECTION

# =====

security_logs = []

def analyze_traffic(row):

    source_ip = row["source_ip"]

    request_count = row["requests"]

```

```

alerts = []

# -----
# Check IP address
# -----

if source_ip not in trusted_ips:
    alerts.append("POSSIBLE IP SPOOFING")
# -----

# Check request volume
# -----

if request_count > DOS_THRESHOLD:
    alerts.append("POSSIBLE DoS")
# -----

# Final decision
# -----

if len(alerts) == 0:
    decision = "NORMAL"
else:
    decision = "BLOCK / INVESTIGATE"

security_logs.append({
    "time": datetime.now().strftime("%H:%M:%S"),
    "source_ip": source_ip,
    "destination_ip": row["destination_ip"],
    "requests": request_count,
    "alert": ", ".join(alerts) if alerts else "NONE",
    "decision": decision
})

```

```

    return decision

# =====

# 8. ANALYZE EVERY TRAFFIC RECORD

# =====

for index, row in df.iterrows():

    analyze_traffic(row)

logs_df = pd.DataFrame(security_logs)

# =====

# 9. DISPLAY SECURITY LOGS

# =====

print("\n" + "=" * 110)

print("NETWORK SECURITY LOG")

print("=" * 110)

print(logs_df.to_string(index=False))

# =====

# 10. DETECT DoS ATTACKS

# =====

dos_alerts = logs_df[

    logs_df["alert"].str.contains("DoS", na=False)

]

print("\n" + "=" * 80)

print("DoS ATTACK DETECTION")

print("=" * 80)

print("Number of Possible DoS Records:", len(dos_alerts))

if len(dos_alerts) > 0:

    print("\nPossible DoS traffic detected:")

```

```

print(
    dos_alerts[
        ["source_ip", "destination_ip", "requests", "alert"]
    ].to_string(index=False)
)
else:
    print("No possible DoS traffic detected.")

# =====

# 11. DETECT SPOOFING

# =====

spoofing_alerts = logs_df[
    logs_df["alert"].str.contains("SPOOFING", na=False)
]

print("\n" + "=" * 80)

print("IP SPOOFING DETECTION")

print("=" * 80)

print("Number of Possible Spoofing Records:",
      len(spoofing_alerts))

if len(spoofing_alerts) > 0:

    print("\nPossible spoofed traffic detected:")

    print(
        spoofing_alerts[

```

```

        ["source_ip", "destination_ip", "requests", "alert"]
    ].to_string(index=False)
)

else:

    print("No possible spoofing traffic detected.")

# =====

# 12. TRAFFIC CLASSIFICATION

# =====

normal_count = len(
    logs_df[logs_df["decision"] == "NORMAL"]
)

blocked_count = len(
    logs_df[logs_df["decision"] == "BLOCK / INVESTIGATE"]
)

total_count = len(logs_df)

print("\n" + "=" * 80)
print("TRAFFIC CLASSIFICATION")
print("=" * 80)

```

```
print("Total Records    :", total_count)

print("Normal Records   :", normal_count)

print("Suspicious Records :", blocked_count)
```

```
if total_count > 0:
```

```
    print(
        "Normal Percentage  :",
        round((normal_count / total_count) * 100, 2),
        "%"
    )
```

```
    print(
        "Suspicious Percentage:",
        round((blocked_count / total_count) * 100, 2),
        "%"
    )
```

```
# =====

# 13. SOURCE IP ANALYSIS

# =====
```

```
print("\n" + "=" * 80)

print("SOURCE IP ANALYSIS")

print("=" * 80)
```

```

ip_counts = Counter(df["source_ip"])

for ip, count in ip_counts.items():

    status = "TRUSTED"

    if ip not in trusted_ips:
        status = "UNTRUSTED / SUSPICIOUS"

    print(
        f"{ip:<18} Records: {count:<5} Status: {status}"
    )

# =====

# 14. VISUALIZE REQUEST VOLUME

# =====

plt.figure(figsize=(12, 6))

plt.bar(
    range(len(df)),
    df["requests"]
)

plt.axhline(
    y=DOS_THRESHOLD,

```

```
linestyle="--",  
label="DoS Detection Threshold"  
)  
  
plt.xlabel("Traffic Record")  
plt.ylabel("Number of Requests")  
plt.title("Simulated Network Traffic and DoS Detection Threshold")  
  
plt.legend()  
  
plt.show()
```

```
# =====  
# 15. VISUALIZE NORMAL VS SUSPICIOUS TRAFFIC  
# =====
```

```
labels = ["Normal", "Suspicious"]  
values = [normal_count, blocked_count]  
  
plt.figure(figsize=(7, 5))  
  
plt.bar(labels, values)  
  
plt.xlabel("Traffic Classification")  
plt.ylabel("Number of Records")  
plt.title("Normal vs Suspicious Network Traffic")
```



```
plt.show()
```

```
# =====
```

```
# 16. FINAL SECURITY SUMMARY
```

```
# =====
```

```
print("\n" + "=" * 80)
```

```
print("FINAL SECURITY SUMMARY")
```

```
print("=" * 80)
```

```
print("Protected Server :", server_ip)
```

```
print("Trusted IPs      :", len(trusted_ips))
```

```
print("Total Records   :", total_count)
```

```
print("Normal Traffic  :", normal_count)
```

```
print("Suspicious     :", blocked_count)
```

```
print("DoS Alerts      :", len(dos_alerts))
```

```
print("Spoofing Alerts :", len(spoofing_alerts))
```

```
print("\nRecommended Action:")
```

```
print("Suspicious traffic should be blocked or investigated.")
```

```
print("\n" + "=" * 80)
```

```
print("PRACTICAL 9 COMPLETED SUCCESSFULLY")
```

```
print("=" * 80)
```

```
print("""
```

The controlled simulation demonstrated:

- [1] Normal network traffic
- [2] Simulated DoS traffic
- [3] Simulated IP spoofing
- [4] Request-volume analysis
- [5] Source-IP validation
- [6] DoS threshold detection
- [7] Spoofing detection
- [8] Security logging
- [9] Traffic classification
- [10] Traffic statistics
- [11] Attack visualization
- [12] Basic mitigation decision

RESULT:

Network attacks were successfully simulated and detected in a controlled Python environment without generating real attack traffic.

```
""")
```

```
print("=" * 80)
```

10. Expected Output

After execution, students will observe output similar to:

=====

SIMULATION OF NETWORK ATTACKS

CONTROLLED DoS AND IP SPOOFING SIMULATION

=====

Trusted IP Addresses:

- 10.0.0.10
- 10.0.0.11
- 10.0.0.12
- 10.0.0.13

Protected Server: 10.0.0.50

DoS Detection Threshold: 50

Total Simulated Traffic Records: 46

The security log will contain entries such as:

source_ip	requests	alert
-----------	----------	-------

10.0.0.11	12	NONE
10.0.0.10	8	NONE
10.0.0.99	125	POSSIBLE DoS
192.168.1.200	21	POSSIBLE IP SPOOFING
172.16.0.25	18	POSSIBLE IP SPOOFING
10.0.0.99	140	POSSIBLE DoS

The program will then produce:

DoS ATTACK DETECTION

Number of Possible DoS Records: 8

and:

IP SPOOFING DETECTION

Number of Possible Spoofing Records: 8

The graphs will show the difference between normal traffic and traffic exceeding the simulated DoS threshold.

11. Practical Tasks for Students

After running the basic program, students should perform the following modifications.

Task 1 – Change DoS Threshold

Change:

DOS_THRESHOLD = 50

to:

DOS_THRESHOLD = 30

Observe how the number of detected DoS records changes.

Task 2 – Add a Trusted IP

Add:

"10.0.0.14"

to the trusted IP list.

Run the program again and observe the result.

Task 3 – Add a New Spoofed IP

Add:

"203.0.113.50"

to the `spoofed_ips` list.

Observe the security log.

Task 4 – Analyze Attack Sources

Identify:

- Most frequent source IP
- Number of DoS alerts
- Number of spoofing alerts
- Number of normal records
- Number of suspicious records

Task 5 – Modify the Visualization

Modify the graph to compare:

Normal Traffic

DoS Traffic

Spoofed Traffic

12. Observation Table

Sr. No.	Source IP	Request s	IP Status	Detectio n	Final Decision
1	10.0.0.10	12	Trusted	Normal	Normal
2	10.0.0.11	18	Trusted	Normal	Normal
3	10.0.0.99	125	Untruste d	DoS	Block/Investigat e
4	192.168.1.20 0	20	Untruste d	Spoofing	Block/Investigat e
5	10.0.0.12	10	Trusted	Normal	Normal

6	172.16.0.25	25	Untrusted	Spoofing	Block/Investigate
---	-------------	----	-----------	----------	-------------------

The actual values will vary because the program generates traffic randomly.

13. Result

Result: The controlled network attack simulation was successfully implemented in Python using Google Colab. Simulated DoS traffic was identified using request-volume thresholds, while possible IP spoofing was detected by validating source IP addresses against a trusted IP list. Security logs, traffic classification, statistics, and graphical analysis were successfully generated.

14. Conclusion

This practical demonstrated how basic network attacks can be analyzed in a controlled environment. The simulation showed that abnormal request volumes can indicate a possible **DoS attack**, while unexpected source addresses can indicate possible **IP spoofing**.